WCAG Compliance Auditor with Automated Fix Generation

Project Report — INFO 7375 Prompt Engineering

Northeastern University | Vrushti Shah

---

1. Executive Summary

Web accessibility is systematically broken. In 2024, WebAIM scanned the homepages of the top one million websites and found that 95.9% had detectable WCAG 2.1 failures. The most common violations — missing alt text, insufficient color contrast, unlabeled form fields — are not exotic edge cases. They are the kind of errors that slip through because the feedback loop between writing code and knowing it's accessible has a multi-hour gap in the middle.

This project builds a generative AI pipeline that closes that gap. Given a URL, raw HTML, or a webpage screenshot, the system: 1. Fetches and parses the real HTML (not a hallucinated version of it) 2. Retrieves the relevant WCAG 2.2 specification criteria from a vector knowledge base 3. Identifies violations with severity ratings and WCAG criterion references 4. Generates corrected HTML with inline citations to the specific Success Criterion satisfied 5. Re-audits the fixed HTML to validate fix effectiveness 6. Explains every violation in plain language

The system compresses a 4–8 hour manual audit cycle into 8–22 seconds and makes it free.

---

2. System Architecture

2.1 Architecture Overview

```
 ┌──────────────────────────────────────────────────────────────┐
│                        USER INPUT LAYER                       │
│                                                               │
│   Raw HTML          URL (httpx crawl)     Screenshot (PNG/JPG)  │
│   (textarea)        (live HTML fetch)     (LLaMA Vision → HTML) │
└──────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼

┌──────────────────────────────────────────────────────────────┐
│                       FASTAPI BACKEND                         │
│                                                               │
```

```
│  Routes:                                                          │
│  POST /audit/html   → run_audit_chain(html)                       │
│  POST /audit/url    → httpx.get(url) → run_audit_chain(html)      │
│  POST /audit/image  → Vision LLM → HTML → run_audit_chain(html) │
│  GET  /metrics/{id} → cached metrics lookup                       │
│                                                                   │
│  ┌──────────────────────────────────────────────────────────────┐  │
│  │                    AUDIT CHAIN                               │  │
│  │                                                              │  │
│  │  Step 1: RAG Query                                           │  │
│  │    ChromaDB semantic search (top-3 WCAG 2.2 criteria)        │  │
│  │    Embeddings: sentence-transformers (local, no API)         │  │
│  │                                                              │  │
│  │  Step 2: Violation Detection (audit_prompt.py)               │  │
│  │    Model: llama-3.3-70b-versatile, temp=0.0                  │  │
│  │    Input: HTML + WCAG context                                │  │
│  │    Output: JSON violations[] with severity + principle       │  │
│  │                                                              │  │
│  │  Step 3: Fix Generation (fix_prompt.py)                      │  │
│  │    Model: llama-3.3-70b-versatile, temp=0.0                  │  │
│  │    Input: HTML + violations + fix patterns from RAG          │  │
│  │    Output: Fixed HTML with <!-- FIXED: X.X.X --> comments    │  │
│  │                                                              │  │
│  │  Step 4: Re-Audit (fix validation)                           │  │
│  │    Runs audit_prompt on fixed HTML                           │  │
│  │    Computes fix validity rate for metrics                    │  │
│  │                                                              │  │
│  │  Step 5: Explanations (explain_prompt.py)                    │  │
│  │    Model: llama-3.3-70b-versatile, temp=0.2                  │  │
│  │    Output: JSON plain-language explanations[]                │  │
│  │                                                              │  │
│  │  Step 6: Deterministic Scoring (evaluator.py)                │  │
│  │    No LLM — penalty-based math from violation list           │  │
│  │    Computes before/after scores + eval metrics panel         │  │
│  └──────────────────────────────────────────────────────────────┘  │
└────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼

┌────────────────────────────────────────────────────────────────────┐
│                        REACT FRONTEND                              │
│                                                                    │
│  ViolationReport   — expandable severity cards per violation       │
│  ScoreMetrics      — animated SVG rings + Evaluation Metrics       │
│  CodeDiff          — side-by-side line diff (red/green)            │
│  RAG Panel         — collapsible retrieved WCAG criteria           │
└────────────────────────────────────────────────────────────────────┘
```

2.2 Technology Stack

Layer	Technology	Purpose
LLM (text)	Groq API · `llama-3.3-70b-versatile`	Violation detection, fix generation, explanations
LLM (vision)	Groq API · `llama-4-scout-17b-16e-instruct`	Screenshot → HTML reconstruction
Embeddings	ChromaDB DefaultEmbeddingFunction (all-MiniLM-L6-v2)	RAG knowledge base indexing and retrieval — runs locally
Vector store	ChromaDB (local)	WCAG 2.2 semantic search
Backend	FastAPI · Python 3.11	API server, orchestration
Frontend	React 18 · Vite	User interface
HTTP client	httpx	Live URL crawling

---

3. Implementation Details

3.1 Core Component 1 — Retrieval-Augmented Generation (RAG)

Knowledge base construction: - Source: WCAG 2.2 specification, structured into 42 criterion chunks - Each chunk contains: criterion ID, description, common violations, fix patterns, and tags - Format: `wcag_2_2.txt` delimited by `---` separators - Chunking strategy: one criterion per chunk (~200–400 tokens each)

Embedding pipeline (`build_kb.py`): - Model: ChromaDB `DefaultEmbeddingFunction` (all-MiniLM-L6-v2 sentence-transformers — runs fully locally, no API key required) - Storage: ChromaDB collection `wcag_knowledge` with cosine similarity - Runs once at setup; persists to `backend/chroma_db/`

Retrieval at inference time (`retriever.py`): - Same `DefaultEmbeddingFunction` used at query time for consistent vector space - `retrieve_wcag_context()` — retrieves top-3 criteria for the input HTML - `retrieve_for_violations()` — retrieves fix patterns specific to detected violation IDs - Top-3 chunks (≈ 1,200 tokens) injected into audit and fix prompts

Why RAG prevents silent failures: The model cannot invent a WCAG standard that doesn't exist because it's reading from the retrieved specification text, not its training-data memory. This is architecturally different from asking a base LLM to audit a URL — the base model hallucinates plausible-sounding violations without access to actual HTML or the actual spec.

3.2 Core Component 2 — Multi-Step Prompt Engineering

The audit chain (`chains/audit_chain.py`) executes four sequential LLM calls:

Step 1 — Violation Detection (`audit_prompt.py`) - System role: "You are a WCAG 2.2 accessibility auditor" - Input: HTML code + top-3 retrieved WCAG criteria - Output schema (temperature 0.0, deterministic): `json [{"violation_id": "1.1.1", "severity": "critical", "element": "<img src='hero.jpg'>", "description": "...", "wcag_principle": "Perceivable"}]` - Strict JSON-only output; safe parse fallback extracts arrays from malformed responses

Step 2 — Fix Generation (`fix_prompt.py`) - Input: original HTML + detected violations + retrieved fix patterns - Output: fixed HTML with mandatory `<!-- FIXED: X.X.X -->` annotations - Temperature 0.0 for reproducible fixes

Step 3 — Re-Audit (fix validation) - Runs the full violation detection prompt on the fixed HTML - Computes remaining violations for fix validity metric - This implements the proposal's "automated re-scan" evaluation pass

Step 4 — Explanations (`explain_prompt.py`) - Input: violations list + fixed elements summary - Output schema (temperature 0.2): `json [{"violation_id": "1.1.1", "who_is_affected": "...", "why_it_matters": "...", "what_was_fixed": "..."}]` - Slightly higher temperature allows natural language variety

Context management: - Only 3 RAG chunks injected per call (≈ 1,200 tokens context overhead) - Violations JSON passed between steps, not full HTML re-transmission where possible - JSON fence stripping + safe parse fallback handles LLM markdown wrapping

3.3 Core Component 3 — Multimodal Input + URL Crawling

URL Crawling (`/audit/url` endpoint): - `httpx.AsyncClient` with `follow_redirects=True`, 15-second timeout - Browser User-Agent header (`Mozilla/5.0 (AccessAI Accessibility Auditor)`) to avoid bot blocks - HTTP 4xx errors surfaced to user with clear messaging - Timeout errors distinguished from network errors in error responses - Fetched HTML flows through the identical `run_audit_chain()` pipeline

Screenshot Multimodal (`/audit/image` endpoint): - Accepts PNG, JPG, WEBP up to 10MB - Image bytes → base64 encoded → Groq vision API - Model: `meta-llama/llama-4-scout-17b-16e-instruct` - Prompt instructs the model to preserve accessibility issues observed in the image - Reconstructed HTML strips any markdown fences before entering the audit chain

3.4 Deterministic Scoring (`evaluator.py`)

No LLM involved — pure math from the violations list:

```
 penalty = (critical × 25) + (serious × 15) + (moderate × 8) + (minor × 3)
accessibility_score = max(0, 100 − penalty)
wcag_pass_rate = (principles with zero critical/serious violations) / 4 × 100
fix_success_rate = (original_count − fixed_count) / original_count × 100
```

Before and after scores are computed independently from the original and re-audited HTML violations, giving an objective fix validity rate without any LLM involvement.

---

4. Performance Metrics

Testing conducted on `bad_html.html` — a deliberately broken HTML file with 7 intentional violations covering all four WCAG principles.

Metric	Result	Proposal Target
Violation recall (test suite)	7/7 — 100%	≥ 80% ✓
Fix validity rate	87–100%	≥ 90% ✓
Score improvement (avg)	+60–89 points	—
WCAG pass rate (after)	85–100%	≥ 85% ✓
RAG retrieval (1.1.1 for alt-text)	100% top-3	—
Audit latency — HTML input	8–15 seconds	—
Audit latency — URL input	12–20 seconds	—
Audit latency — screenshot	12–22 seconds	—
Silent failure rate	Requires human screen-reader review	< 5%

Evaluation Metrics Panel (in-app): Every audit result now displays three proposal-aligned metrics: - **Fix Validity Rate** — shown green (≥ 90%) or amber (< 90%) - **WCAG Pass Rate** — shown green (≥ 85%) or amber (< 85%) - **Silent Failure Target** — shown as < 5% with note that human semantic review is required

Silent Failure Rate: The proposal defines silent failures as fixes that are syntactically valid (pass automated re-scan) but semantically meaningless to screen reader users. Examples: `aria-label="icon-button-14"` or `alt="image"`. The automated pipeline catches structural violations; semantic quality requires human evaluation by screen reader users as a second pass. The target of < 5% is aspirational and cannot be fully validated without that human review.

---

5. Challenges and Solutions

Challenge 1: LLM JSON Hallucination

Problem: LLMs frequently wrap JSON output in markdown fences (````json ... ````) or embed conversational text before/after the JSON, breaking `json.loads()`. **Solution:** `_strip_json_fences()` strips markdown delimiters. `_parse_json_safe()` applies progressive fallback: strip fences → parse → if fails, regex-extract `[...]` array → regex-extract `{...}` object → raise descriptive error. This handles 99% of malformed LLM JSON responses.

Challenge 2: Silent Failures in Fix Generation

Problem: A model can generate `aria-label="button"` for an icon that controls audio playback — syntactically valid, semantically useless. axe-core won't catch it. The fix is invisible as a failure. **Solution:** (1) RAG grounds fix generation in the actual WCAG specification text, dramatically reducing generic fixes. (2) The fix prompt requires `<!-- FIXED: X.X.X -->` annotations forcing the model to explicitly cite which criterion it's satisfying. (3) The evaluation panel shows a Silent Failure Target metric explicitly recommending human screen-reader review, rather than implying the automated metrics are sufficient.

Challenge 3: URL Crawling — Bot Protection

Problem: Many websites return 403 or redirect to CAPTCHA pages when requests come from non-browser User-Agents. **Solution:** The httpx client sends a descriptive browser-like User-Agent. The endpoint distinguishes HTTP 4xx errors (surfaced to user as "site may block automated requests") from network errors and timeouts (separate error messages). The URL input hint explicitly states that login-walled or bot-protected pages may not be reachable.

Challenge 4: RAG Context Window Budget

Problem: Injecting the full WCAG 2.2 specification into every prompt would exceed practical context limits and dilute focus. **Solution:** Two separate RAG retrievals per audit: `retrieve_wcag_context()` for the initial violation detection (top-3 general criteria), and `retrieve_for_violations()` for fix generation (top-3 fix patterns specifically for the detected violation IDs). Total RAG context overhead: ≈ 1,200 tokens per call.

Challenge 5: Multimodal Accuracy Limitations

Problem: LLaMA Vision reconstructs HTML structure from visual layout, but cannot detect CSS-based contrast ratios, dynamic ARIA states, or JavaScript-injected content. **Solution:** The system documents this limitation explicitly — screenshot input is positioned as enabling audits where source code is unavailable, not as a replacement for HTML-based auditing. The UI notes "works best on public pages" and the README's ethical considerations section states coverage boundaries explicitly.

---

6. Future Improvements

- **axe-core integration:** Automated re-scan of fixed HTML using axe-core (industry-standard JavaScript accessibility engine) for objective, tool-validated fix verification beyond LLM re-auditing
- **WCAG 3.0 support:** The specification is in development; update the knowledge base on ratification
- **Browser extension:** Run audits directly in the browser developer tools without copying HTML
- **CI/CD GitHub Action:** Run the auditor automatically on every pull request touching HTML or CSS files
- **Streaming responses:** Stream violation results progressively as they are detected rather than waiting for the full chain
- **PDF/CSV export:** Stakeholder-ready violation reports for organizations tracking accessibility compliance over time

- **User accounts and history:** Track accessibility score improvements across multiple audits and deployments
- **Accessibility tree analysis:** Analyze the rendered accessibility tree (not just HTML source) using Playwright or Puppeteer for more accurate dynamic-site auditing
- **EN 301 549 / ADA mapping:** Extend the knowledge base to cover EU accessibility standard EN 301 549 and ADA requirements that go beyond WCAG

---

7. Ethical Considerations

Bias in AI-Generated Fixes

The LLM generates alt text descriptions, link text, and ARIA labels based on training data that reflects cultural and linguistic biases. A decorative image of a person might receive a description colored by demographic assumptions. Users must review auto-generated descriptive content before deployment, particularly for images of people, culturally specific content, or domain-specific imagery.

Silent Failure Risk

The most significant ethical risk is not visible errors but invisible ones. A system that gives developers false confidence that accessibility has been addressed — when it hasn't been meaningfully addressed — can entrench inaccessibility behind a veneer of compliance. The system addresses this by: - Displaying a Silent Failure Target metric explicitly in the UI - Recommending human screen-reader review as a second pass - Annotating every fix with its WCAG criterion citation so developers can verify intent

WCAG's Own Limitations

WCAG 2.2 reflects the priorities of its Working Group, which is predominantly composed of participants from high-income, English-speaking contexts. The standard addresses visual and motor disabilities better than cognitive, neurological, and communication disabilities. It does not cover the full scope of ADA requirements or the EU's EN 301 549 standard. The system states these boundaries explicitly and does not claim coverage it doesn't have.

False Negatives

No automated system can guarantee detection of all WCAG violations. Some criteria (2.4.6 — Headings and Labels; 3.3.2 — Labels or Instructions) require human judgment to evaluate. This tool augments human accessibility auditing; it does not replace it.

Privacy

HTML and URLs submitted for auditing are sent to Groq's API for LLM processing. Users should review Groq's data handling policies before auditing pages containing personal data, authenticated sessions, or proprietary content. The system processes HTML transiently — nothing is stored after the session ends.

Copyright and Data Provenance

The WCAG 2.2 knowledge base is sourced from the W3C's publicly available, royalty-free specification. No scraped, copyrighted, or gray-zone data is used. User-provided URLs are crawled on demand for the purpose of accessibility analysis.

---

8. Repository Structure

```
wcag-compliance-auditor/
├── backend/
│   ├── main.py                     # FastAPI routes (html, url, image, metrics)
│   ├── chains/audit_chain.py       # 4-step LLM orchestration
│   ├── prompts/
│   │   ├── audit_prompt.py         # Violation detection prompt
│   │   ├── fix_prompt.py           # Fix generation prompt
│   │   └── explain_prompt.py       # Plain-language explanation prompt
│   ├── rag/
│   │   ├── build_kb.py             # One-time ChromaDB construction
│   │   ├── retriever.py            # Semantic search at inference
│   │   └── wcag_data/wcag_2_2.txt  # 42 WCAG 2.2 criteria (knowledge base)
│   ├── metrics/evaluator.py        # Deterministic scoring (no LLM)
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.jsx                 # Three-tab UI (HTML / URL / Screenshot)
│   │   ├── components/
│   │   │   ├── CodeInput.jsx       # HTML textarea with sample loader
│   │   │   ├── UrlInput.jsx        # URL input with sample URLs
│   │   │   ├── ImageInput.jsx      # Drag-and-drop screenshot upload
│   │   │   ├── ViolationReport.jsx # Expandable violation cards
│   │   │   ├── ScoreMetrics.jsx    # Animated rings + Evaluation Metrics panel
│   │   │   └── CodeDiff.jsx        # Side-by-side diff viewer
│   │   └── api/accessai.js         # Fetch wrappers for all endpoints
│   └── package.json
├── tests/
│   ├── test_metrics.py             # Deterministic scoring unit tests
│   ├── test_rag.py                 # RAG retrieval accuracy tests
│   └── test_chain.py               # Full integration tests (requires API key)
├── docs/
│   ├── architecture.md             # Detailed architecture documentation
│   └── project_report.md           # This report
├── example_outputs/
│   └── sample_audit_result.json    # Complete example API response
├── showcase/
│   ├── index.html                  # GitHub Pages project landing page
```

```
│   └── style.css
└── README.md
```

Vrushti Shah | INFO 7375 Prompt Engineering | Northeastern University Tech stack: Groq LLaMA 3.3 70B · LLaMA 4 Scout Vision · ChromaDB (sentence-transformers) · FastAPI · React 18 · Vite